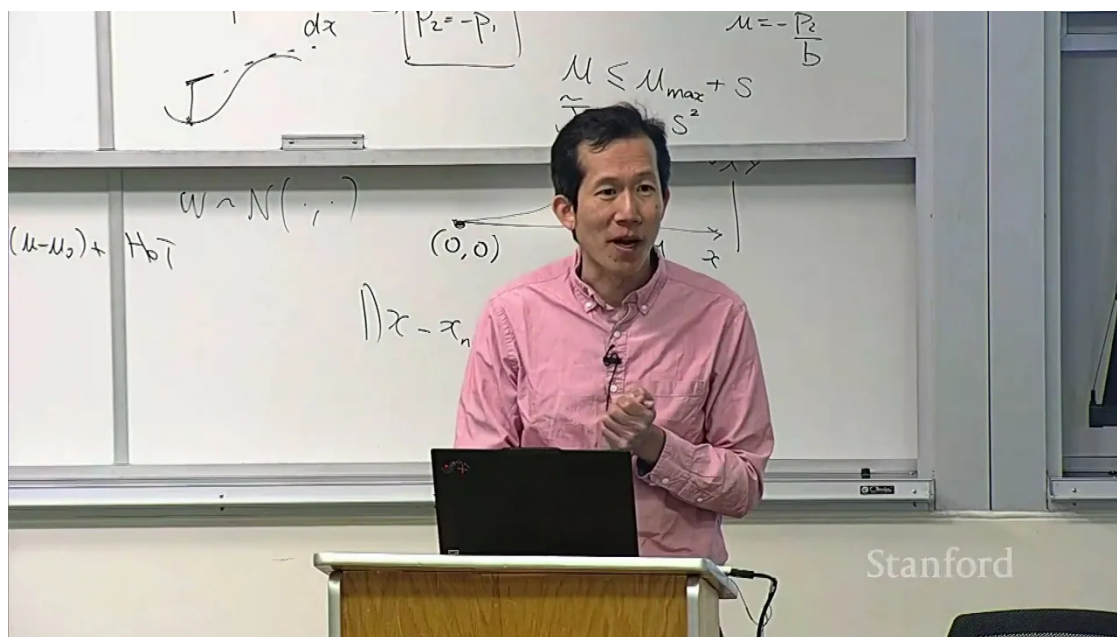


Stanford CS336: Language Modeling from Scratch

Lecture 6: Kernels, Triton, and XLA

Spring 2026 中文精讲笔记

主讲: Percy Liang



频道: Stanford Online

发布日期: 2026-04-28

时长: 1:26:41

视频: <https://www.youtube.com/watch?v=xnDHaNUvHBg>

官方主页: <https://cs336.stanford.edu/>

目录

1 导读：问题、证据与阅读方法	2
2 GPU 拓扑和线程层级	2
3 warp 占用率与共享内存	7
4 正确的内核基准方法	11
5 从 CUDA 到 Triton	15
6 融合 softmax	19
7 并行归约	23
8 分块矩阵乘	27
9 编译器 XLA 与优化边界	31
10 代码化复现：最小机制	35
11 总结与延伸	36
12 资料来源与审计边界	36

1 导读：问题、证据与阅读方法

本讲主问题

怎样从算子语义出发，写出既正确又能利用 GPU 层级并行和存储复用的 CUDA/Triton 内核？

内核优化是一条可验证链：定义索引与掩码，测量真实时间，定位访存或计算瓶颈，再改变映射和融合方式。本笔记以课程视频、人工字幕和 Stanford 官方讲义或代码为共同证据源。图像均来自每十五秒抽帧后的人工接触表审查，时间脚注可回到原视频复核；公式区分课程机制关系和本笔记的决策分析框架。学习时建议先读主问题，再沿“机制-资源-失败模式-验证指标”四列整理自己的实验。

首先排除的误读

没有预热与同步的基准、越界掩码错误、bank conflict 和过大的资源占用都会制造看似更快但不可复现的结果。

1.1 关键数字核对

主张	数值或关系	视频时间
NVIDIA warp 的线程数	32	00:15:00
常见 CUDA 线程块上限	1024 threads	00:11:00
共享内存常见 bank 数	32	00:22:00
密集抽帧间隔	15 s	全片

1.2 本章小结

本章建立证据边界：视频解释提供因果和强调，官方材料提供公式与代码，精选帧提供可核查的视觉状态。后文每个主题都同时回答“为何重要、怎样工作、怎样失败、怎样验证”，避免把课程压缩成术语目录。

2 GPU 拓扑和线程层级

本节路线

本节依次处理：GPU 核心存储与互连组成的硬件拓扑、线程线程块和网格的索引层级、CUDA 程序抽象与硬件调度单元的映射、warp 调度与占用率共同隐藏访存延迟。它们共享一个资源与信息流边界，因而必须联合阅读，不能把其中某张图当成独立结论。

$$i = \text{program_id} \cdot B + \text{offset}$$

- 公式中的大写字母表示张量规模、样本规模、设备数或资源总量，具体语义由本节上下文给出。
- 小写字母和希腊字母表示权重、比例、阈值、延迟或拟合参数；下标用于区分样本、设备、专家或时间步。

- 等号给出机制关系，近似号表示课程中的资源估算；实际系统还要用实测常数校准。

$$G_1 = \frac{B_1}{C_1 + \lambda R_1 + \varepsilon}$$

- B_1 是本节方法带来的质量、吞吐或覆盖收益。
- C_1 是新增计算、显存、通信、数据或标注成本。
- R_1 是错误、偏差、失稳或不可复现风险； λ 表示对风险的惩罚强度。

$$C_1 \leq C_{\max}, \quad R_1 \leq R_{\max}, \quad Q_1 \geq Q_{\min}$$

- $C_{\max}$ 和 $R_{\max}$ 分别是资源预算与风险上限。
- $Q_{\min}$ 是必须守住的质量、正确率或服务水平下界。
- 这组约束是本笔记用于复现实验决策的分析框架，并非把课程中的所有方法压缩为单一分数。

2.1 GPU 核心存储与互连组成的硬件拓扑

核心判断。“GPU 核心存储与互连组成的硬件拓扑”是“GPU 拓扑和线程层级”中的一个可检验设计选择，而不是只需记忆的产品名。把算子映射到线程或矩阵单元，并明确数据位于寄存器、片上 SRAM 还是 HBM；瓶颈判断依赖算术强度，而不是只比较峰值 FLOP/s。

复现与诊断。实验记录至少包含输入形状或数据范围、关键状态位置、阈值与精度、资源测量和质量基线。先检查直接中间量，再检查最终指标；若最终分数改变而中间量没有按机制预测变化，应优先怀疑实现、基准或数据泄漏。第 1 个图像证据在本讲 25 个主题中的作用，是把这一判断固定到可回看的原始画面和时间区间。

应记住的判断

围绕“GPU 核心存储与互连组成的硬件拓扑”，至少记录直接机制指标、最终质量指标和一个反事实基线。若只给最终分数而没有中间量，就无法区分真正的机制收益、额外预算和测量误差。

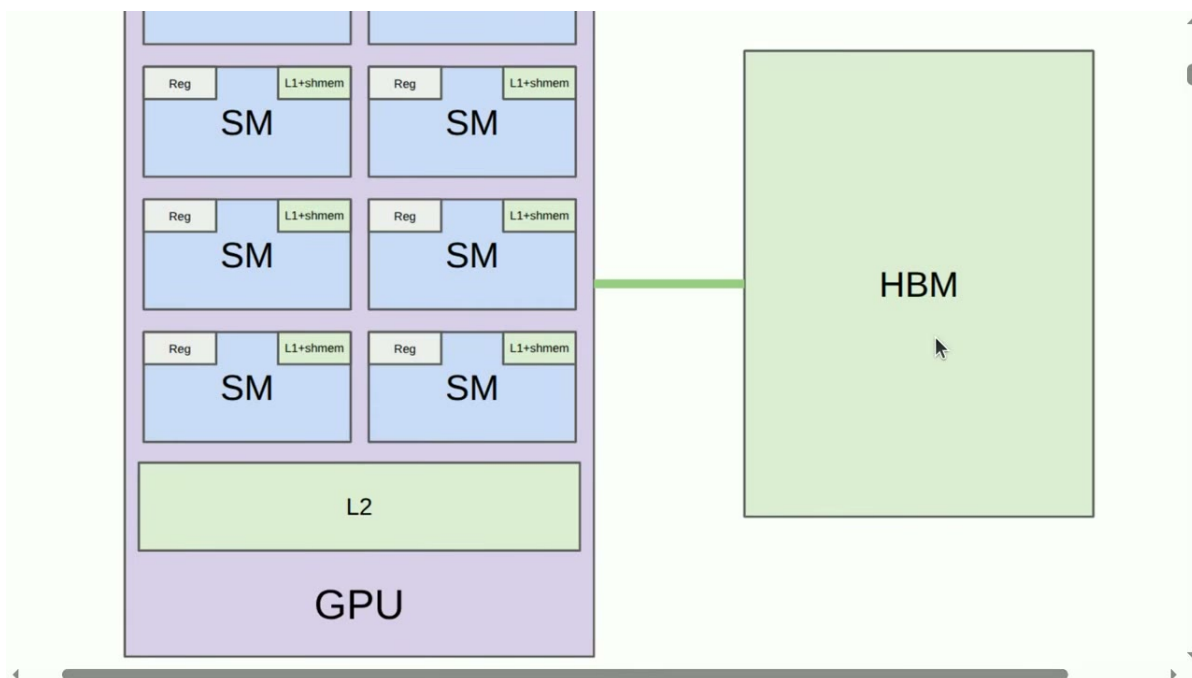


图 1: GPU 核心存储与互连组成的硬件拓扑¹

2.2 线程线程块和网格的索引层级

核心判断。“线程线程块和网格的索引层级”是“GPU 拓扑和线程层级”中的一个可检验设计选择，而不是只需记忆的产品名。把该主题拆成输入、变换、持久状态和输出四部分，再分别记录直接机制指标、资源成本和最终质量；若中间量不按预测变化，应优先检查实现和数据边界。

复现与诊断。实验记录至少包含输入形状或数据范围、关键状态位置、阈值与精度、资源测量和质量基线。先检查直接中间量，再检查最终指标；若最终分数改变而中间量没有按机制预测变化，应优先怀疑实现、基准或数据泄漏。第 2 个图像证据在本讲 25 个主题中的作用，是把这一判断固定到可回看的原始画面和时间区间。

复现实验记录

围绕“线程线程块和网格的索引层级”，至少记录直接机制指标、最终质量指标和一个反事实基线。若只给最终分数而没有中间量，就无法区分真正的机制收益、额外预算和测量误差。

¹视频画面时间区间：00:03:00–00:03:15。

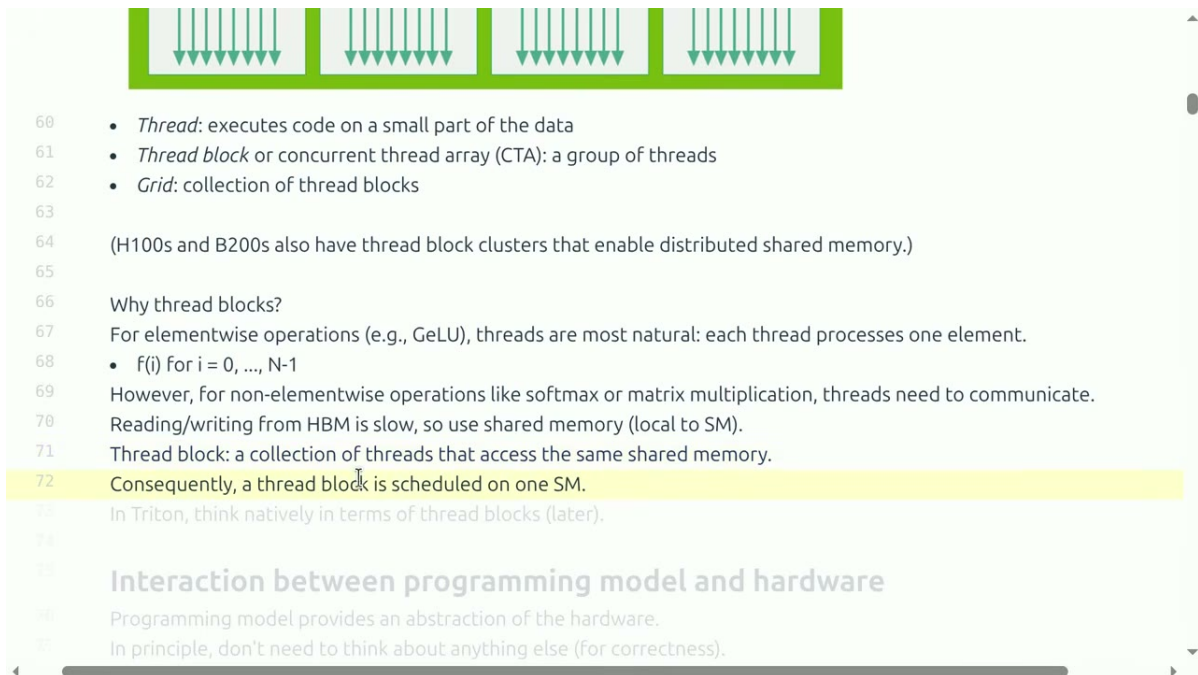


图 2: 线程线程块和网格的索引层级²

2.3 CUDA 程序抽象与硬件调度单元的映射

核心判断。“CUDA 程序抽象与硬件调度单元的映射”是“GPU 拓扑和线程层级”中的一个可检验设计选择，而不是只需记忆的产品名。把算子映射到线程或矩阵单元，并明确数据位于寄存器、片上 SRAM 还是 HBM；瓶颈判断依赖算术强度，而不是只比较峰值 FLOP/s。

复现与诊断。实验记录至少包含输入形状或数据范围、关键状态位置、阈值与精度、资源测量和质量基线。先检查直接中间量，再检查最终指标；若最终分数改变而中间量没有按机制预测变化，应优先怀疑实现、基准或数据泄漏。第 3 个图像证据在本讲 25 个主题中的作用，是把这一判断固定到可回看的原始画面和时间区间。

常见失效模式

围绕“CUDA 程序抽象与硬件调度单元的映射”，至少记录直接机制指标、最终质量指标和一个反事实基线。若只给最终分数而没有中间量，就无法区分真正的机制收益、额外预算和测量误差。

²视频画面时间区间：00:06:00–00:06:15。

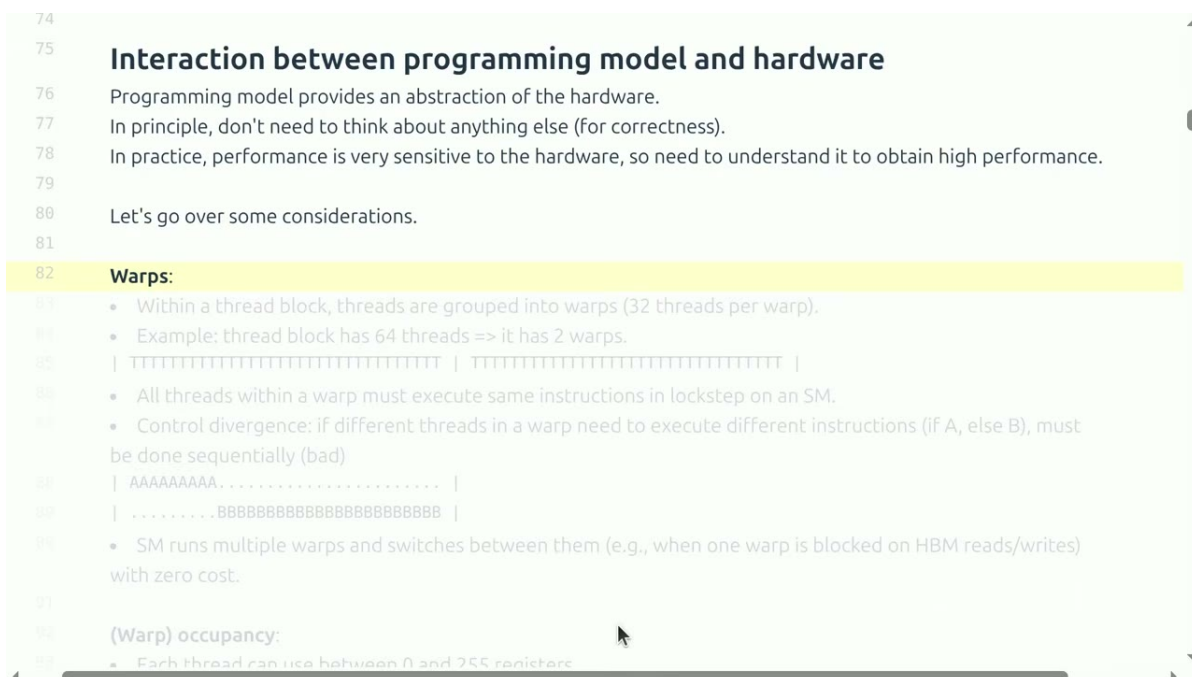


图 3: CUDA 程序抽象与硬件调度单元的映射³

2.4 warp 调度与占用率共同隐藏访存延迟

核心判断。“warp 调度与占用率共同隐藏访存延迟”是“GPU 拓扑和线程层级”中的一个可检验设计选择，而不是只需记忆的产品名。prefill 可以沿序列并行而 decode 逐 token 串行；服务评估要同时报告首 token、token 间隔、吞吐、尾延迟和显存峰值。

复现与诊断。实验记录至少包含输入形状或数据范围、关键状态位置、阈值与精度、资源测量和质量基线。先检查直接中间量，再检查最终指标；若最终分数改变而中间量没有按机制预测变化，应优先怀疑实现、基准或数据泄漏。第 4 个图像证据在本讲 25 个主题中的作用，是把这一判断固定到可回看的原始画面和时间区间。

应记住的判断

围绕“warp 调度与占用率共同隐藏访存延迟”，至少记录直接机制指标、最终质量指标和一个反事实基线。若只给最终分数而没有中间量，就无法区分真正的机制收益、额外预算和测量误差。

³视频画面时间区间：00:09:00–00:09:15。

$$G_2 = \frac{B_2}{C_2 + \lambda R_2 + \epsilon}$$

- B_2 是本节方法带来的质量、吞吐或覆盖收益。
- C_2 是新增计算、显存、通信、数据或标注成本。
- R_2 是错误、偏差、失稳或不可复现风险； λ 表示对风险的惩罚强度。

$$C_2 \leq C_{\max}, \quad R_2 \leq R_{\max}, \quad Q_2 \geq Q_{\min}$$

- $C_{\max}$ 和 $R_{\max}$ 分别是资源预算与风险上限。
- $Q_{\min}$ 是必须守住的质量、正确率或服务水平下界。
- 这组约束是本笔记用于复现实验决策的分析框架，并非把课程中的所有方法压缩为单一分数。

3.1 共享内存 bank conflict 的访问示例

核心判断。“共享内存 bank conflict 的访问示例”是“warp 占用率与共享内存”中的一个可检验设计选择，而不是只需记忆的产品名。把该主题拆成输入、变换、持久状态和输出四部分，再分别记录直接机制指标、资源成本和最终质量；若中间量不按预测变化，应优先检查实现和数据边界。

复现与诊断。实验记录至少包含输入形状或数据范围、关键状态位置、阈值与精度、资源测量和质量基线。先检查直接中间量，再检查最终指标；若最终分数改变而中间量没有按机制预测变化，应优先怀疑实现、基准或数据泄漏。第 5 个图像证据在本讲 25 个主题中的作用，是把这一判断固定到可回看的原始画面和时间区间。

应记住的判断

围绕“共享内存 bank conflict 的访问示例”，至少记录直接机制指标、最终质量指标和一个反事实基线。若只给最终分数而没有中间量，就无法区分真正的机制收益、额外预算和测量误差。

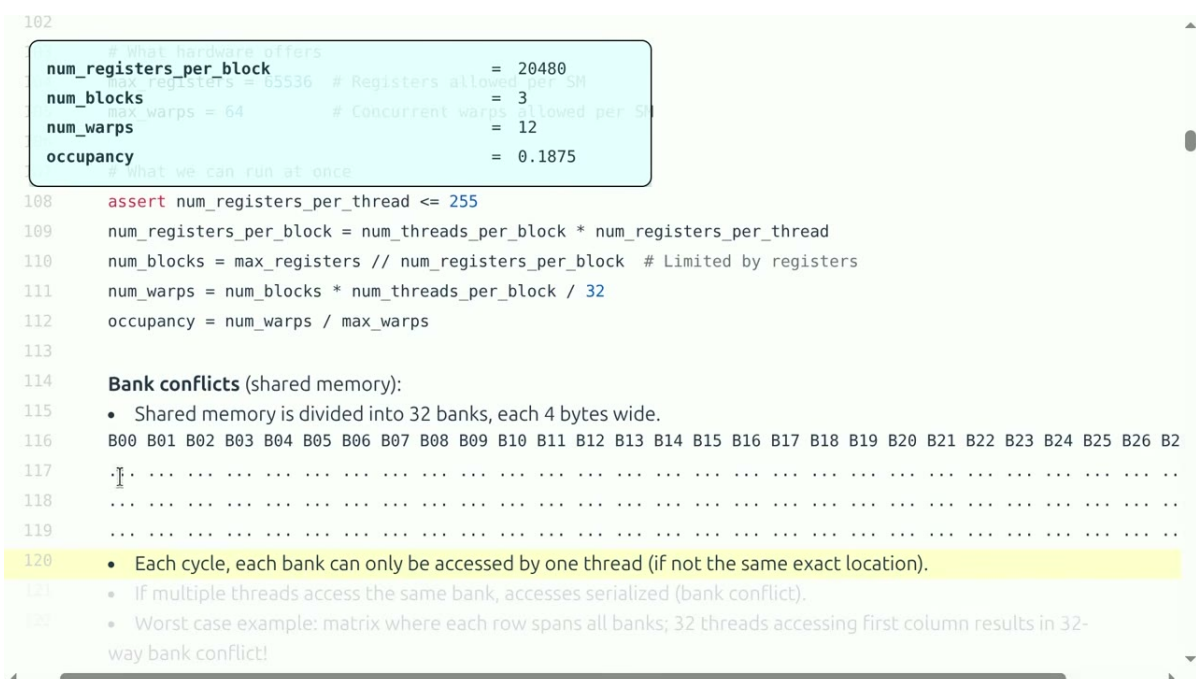


图 5: 共享内存 bank conflict 的访问示例⁵

3.2 bank 映射规则和冲突消除方法

核心判断。“bank 映射规则和冲突消除方法”是“warp 占用率与共享内存”中的一个可检验设计选择，而不是只需记忆的产品名。把该主题拆成输入、变换、持久状态和输出四部分，再分别记录直接机制指标、资源成本和最终质量；若中间量不按预测变化，应优先检查实现和数据边界。

复现与诊断。实验记录至少包含输入形状或数据范围、关键状态位置、阈值与精度、资源测量和质量基线。先检查直接中间量，再检查最终指标；若最终分数改变而中间量没有按机制预测变化，应优先怀疑实现、基准或数据泄漏。第 6 个图像证据在本讲 25 个主题中的作用，是把这一判断固定到可回看的原始画面和时间区间。

复现实验记录

围绕“bank 映射规则和冲突消除方法”，至少记录直接机制指标、最终质量指标和一个反事实基线。若只给最终分数而没有中间量，就无法区分真正的机制收益、额外预算和测量误差。

⁵视频画面时间区间：00:15:00–00:15:15。

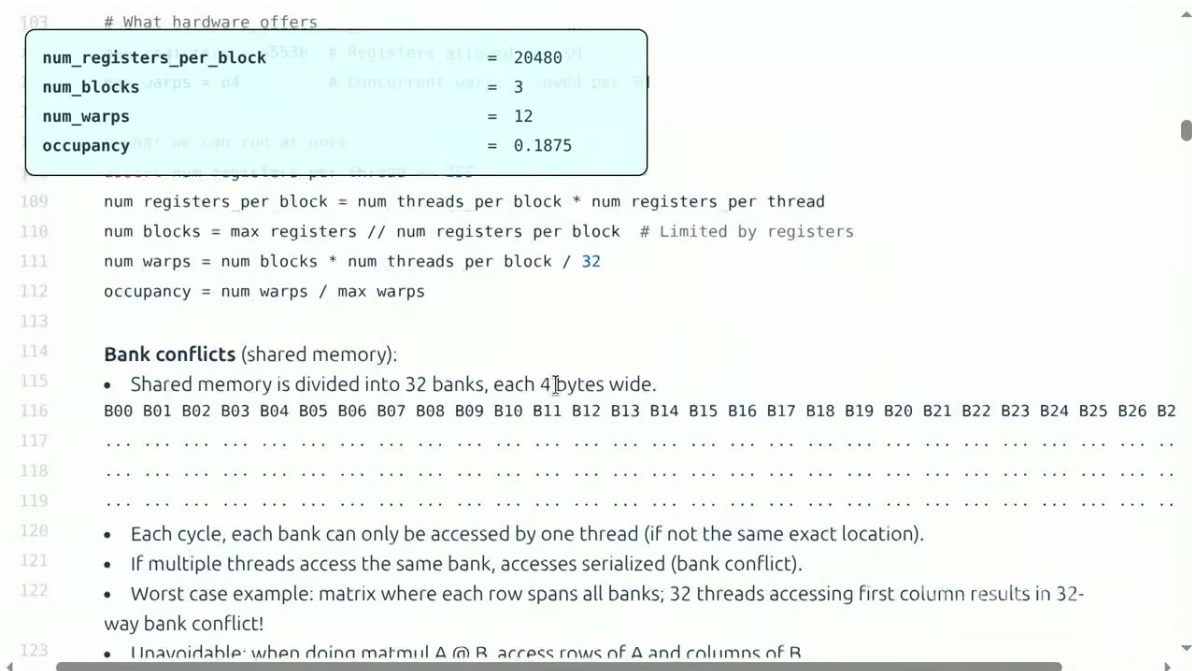


图 6: bank 映射规则和冲突消除方法⁶

3.3 朴素矩阵乘与优化实现的基准差距

核心判断。“朴素矩阵乘与优化实现的基准差距”是“warp 占用率与共享内存”中的一个可检验设计选择，而不是只需记忆的产品名。先用张量形状推出程序实例和掩码，再决定分块、向量化和片上复用；基准必须预热、同步并与高精度参考实现逐元素比较。

复现与诊断。实验记录至少包含输入形状或数据范围、关键状态位置、阈值与精度、资源测量和质量基线。先检查直接中间量，再检查最终指标；若最终分数改变而中间量没有按机制预测变化，应优先怀疑实现、基准或数据泄漏。第 7 个图像证据在本讲 25 个主题中的作用，是把这一判断固定到可回看的原始画面和时间区间。

常见失效模式

围绕“朴素矩阵乘与优化实现的基准差距”，至少记录直接机制指标、最终质量指标和一个反事实基线。若只给最终分数而没有中间量，就无法区分真正的机制收益、额外预算和测量误差。

⁶视频画面时间区间：00:18:00–00:18:15。

```
158     benchmarking measures the wall clock time of performing some operation.  
159     It only gives you end-to-end time, not where time is spent (profiling).  
160  
161     It is still useful for:  
162     • comparing different implementations (which is faster?), and  
163     • understanding how performance scales (e.g., with dimension).  
164  
165     You can use torch.utils.benchmark.  
166     We will roll our own to make benchmarking more transparent.  
167  
168     # Benchmark matrix multiplication  
169     matmul = run_operation2(dim=1024, operation=lambda a, b: a @ b)  
170     result = benchmark(matmul)  
171  
172     # See how timing scales with dimension  
173     results = {}  
174     for dim in [256, 512, 1024, 2048, 4096, 8192]:  
175         results[dim] = benchmark(run_operation2(dim=dim, operation=lambda a, b: a @ b))  
176  
177     Note: time is roughly constant when dimension is small, then cubic scaling.  
178  
179  
180     def benchmark/run: Callable[[Callable, int, int], float]
```

图 7: 朴素矩阵乘与优化实现的基准差距⁷

3.4 本章小结

“warp 占用率与共享内存”的关键不是记住若干实现名称，而是建立选择顺序：先确定目标和状态边界，再核算计算、存储、通信或数据成本，随后检查本节公式的假设，最后用原始画面对应的中间量验证因果链。没有预热与同步的基准、越界掩码错误、bank conflict 和过大的资源占用都会制造看似更快但不可复现的结果。

4 正确的内核基准方法

本节路线

本节依次处理：CUDA 事件计时与预热的基准代码、性能分析器揭示内核时间与数据移动、命令行 profiler 的算子级输出。它们共享一个资源与信息流边界，因而必须联合阅读，不能把其中某张图当成独立结论。

$$\text{occupancy} = \frac{\text{active warps}}{\text{maximum warps}}$$

- 公式中的大写字母表示张量规模、样本规模、设备数或资源总量，具体语义由本节上下文给出。
- 小写字母和希腊字母表示权重、比例、阈值、延迟或拟合参数；下标用于区分样本、设备、专家或时间步。
- 等号给出机制关系，近似号表示课程中的资源估算；实际系统还要用实测常数校准。

⁷视频画面时间区间：00:23:45–00:24:00。

$$G_3 = \frac{B_3}{C_3 + \lambda R_3 + \epsilon}$$

- B_3 是本节方法带来的质量、吞吐或覆盖收益。
- C_3 是新增计算、显存、通信、数据或标注成本。
- R_3 是错误、偏差、失稳或不可复现风险； λ 表示对风险的惩罚强度。

$$C_3 \leq C_{\max}, \quad R_3 \leq R_{\max}, \quad Q_3 \geq Q_{\min}$$

- $C_{\max}$ 和 $R_{\max}$ 分别是资源预算与风险上限。
- $Q_{\min}$ 是必须守住的质量、正确率或服务水平下界。
- 这组约束是本笔记用于复现实验决策的分析框架，并非把课程中的所有方法压缩为单一分数。

4.1 CUDA 事件计时与预热的基准代码

核心判断。“CUDA 事件计时与预热的基准代码”是“正确的内核基准方法”中的一个可检验设计选择，而不是只需记忆的产品名。先用张量形状推出程序实例和掩码，再决定分块、向量化和片上复用；基准必须预热、同步并与高精度参考实现逐元素比较。

复现与诊断。实验记录至少包含输入形状或数据范围、关键状态位置、阈值与精度、资源测量和质量基线。先检查直接中间量，再检查最终指标；若最终分数改变而中间量没有按机制预测变化，应优先怀疑实现、基准或数据泄漏。第 8 个图像证据在本讲 25 个主题中的作用，是把这一判断固定到可回看的原始画面和时间区间。

应记住的判断

围绕“CUDA 事件计时与预热的基准代码”，至少记录直接机制指标、最终质量指标和一个反事实基线。若只给最终分数而没有中间量，就无法区分真正的机制收益、额外预算和测量误差。

```

196     end_event.record() # End timing
197
198     torch.cuda.synchronize() # Wait for CUDA threads to finish
199
200     times.append((start_event.elapsed_time(end_event)))
201
202     mean_time = mean(times)
203     return mean_time
204
205
206 def profiling():
207     While benchmarking looks at end-to-end time, profiling looks at where time is spent.
208     Independent of time, profiling also helps you understand what's going under the hood.
209
210     PyTorch has a built-in profiler.
211     In your assignment, you will use nsight to get more details.
212
213     add(dim=2048)
214     add_profile = profile(run_operation2(dim=2048, operation=lambda a, b: a + b))
215

```

图 8: CUDA 事件计时与预热的基准代码⁸

4.2 性能分析器揭示内核时间与数据移动

核心判断。“性能分析器揭示内核时间与数据移动”是“正确的内核基准方法”中的一个可检验设计选择，而不是只需记忆的产品名。先用张量形状推出程序实例和掩码，再决定分块、向量化和片上复用；基准必须预热、同步并与高精度参考实现逐元素比较。

复现与诊断。实验记录至少包含输入形状或数据范围、关键状态位置、阈值与精度、资源测量和质量基线。先检查直接中间量，再检查最终指标；若最终分数改变而中间量没有按机制预测变化，应优先怀疑实现、基准或数据泄漏。第 9 个图像证据在本讲 25 个主题中的作用，是把这一判断固定到可回看的原始画面和时间区间。

复现实验记录

围绕“性能分析器揭示内核时间与数据移动”，至少记录直接机制指标、最终质量指标和一个反事实基线。若只给最终分数而没有中间量，就无法区分真正的机制收益、额外预算和测量误差。

⁸视频画面时间区间：00:26:45–00:27:00。

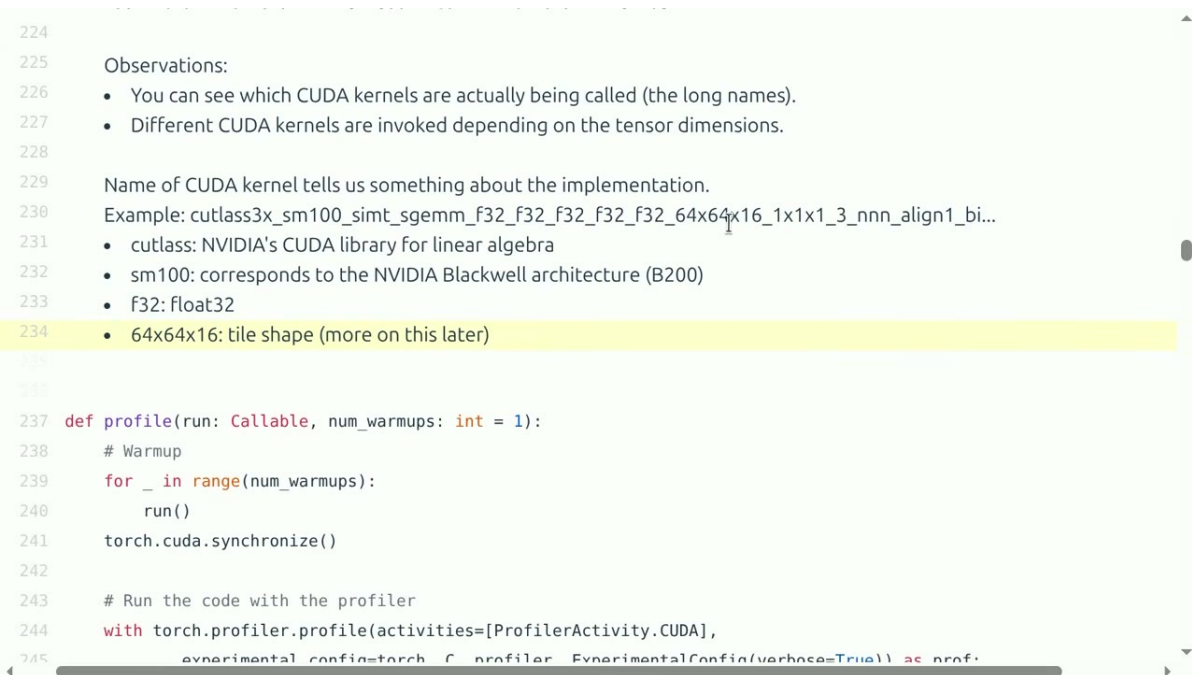


图 9: 性能分析器揭示内核时间与数据移动⁹

4.3 命令行 profiler 的算子级输出

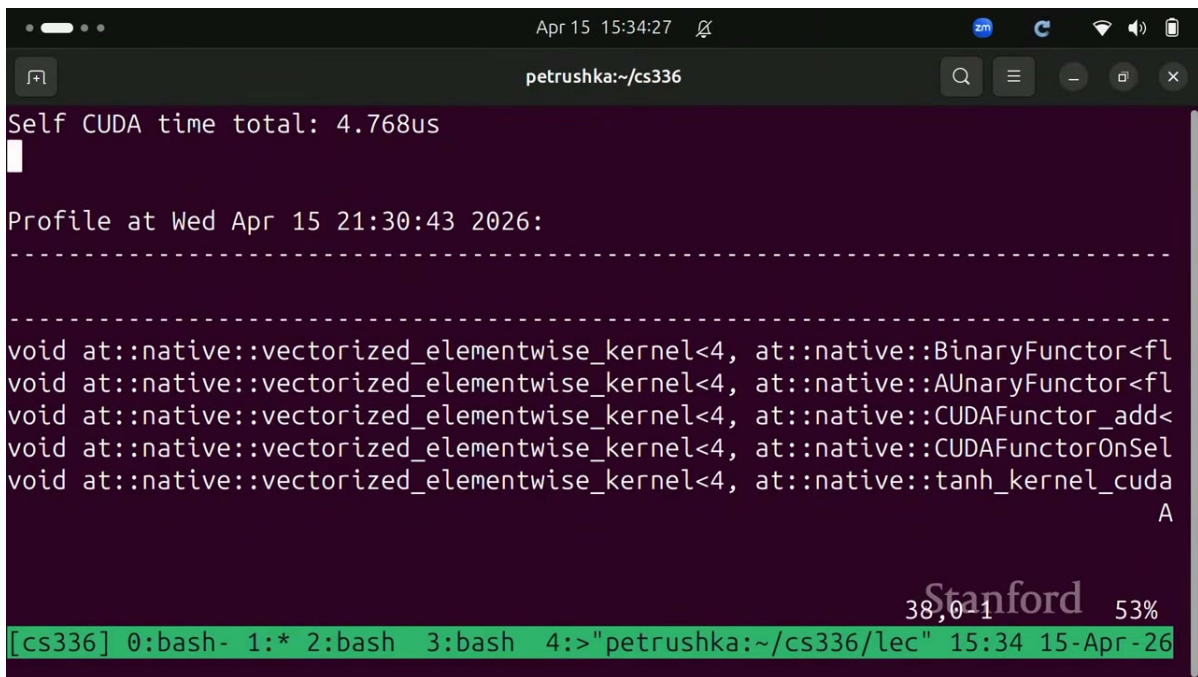
核心判断。“命令行 profiler 的算子级输出”是“正确的内核基准方法”中的一个可检验设计选择，而不是只需记忆的产品名。把该主题拆成输入、变换、持久状态和输出四部分，再分别记录直接机制指标、资源成本和最终质量；若中间量不按预测变化，应优先检查实现和数据边界。

复现与诊断。实验记录至少包含输入形状或数据范围、关键状态位置、阈值与精度、资源测量和质量基线。先检查直接中间量，再检查最终指标；若最终分数改变而中间量没有按机制预测变化，应优先怀疑实现、基准或数据泄漏。第 10 个图像证据在本讲 25 个主题中的作用，是把这一判断固定到可回看的原始画面和时间区间。

常见失效模式

围绕“命令行 profiler 的算子级输出”，至少记录直接机制指标、最终质量指标和一个反事实基线。若只给最终分数而没有中间量，就无法区分真正的机制收益、额外预算和测量误差。

⁹视频画面时间区间：00:29:45–00:30:00。



```
Self CUDA time total: 4.768us

Profile at Wed Apr 15 21:30:43 2026:
-----
void at::native::vectorized_elementwise_kernel<4, at::native::BinaryFunctor<fl
void at::native::vectorized_elementwise_kernel<4, at::native::UnaryFunctor<fl
void at::native::vectorized_elementwise_kernel<4, at::native::CUDataFunctor_add<
void at::native::vectorized_elementwise_kernel<4, at::native::CUDataFunctorOnSel
void at::native::vectorized_elementwise_kernel<4, at::native::tanh_kernel_cuda
A
38,0-1 53%
[cs336] 0: bash- 1:* 2: bash 3: bash 4:> "petrushka:~/cs336/lec" 15:34 15-Apr-26
```

图 10: 命令行 profiler 的算子级输出¹⁰

4.4 本章小结

“正确的内核基准方法”的关键不是记住若干实现名称，而是建立选择顺序：先确定目标和状态边界，再核算计算、存储、通信或数据成本，随后检查本节公式的假设，最后用原始画面对应的中间量验证因果链。没有预热与同步的基准、越界掩码错误、bank conflict 和过大的资源占用都会制造看似更快但不可复现的结果。

5 从 CUDA 到 Triton

本节路线

本节依次处理：CUDA 与 Triton 的抽象层次比较、Triton 程序实例与 block size 的启动语义、Triton 内核中的程序编号掩码和向量化加载。它们共享一个资源与信息流边界，因而必须联合阅读，不能把其中某张图当成独立结论。

$$\text{bank}(a) = \left\lfloor \frac{a}{w} \right\rfloor \bmod N_{\text{bank}}$$

- 公式中的大写字母表示张量规模、样本规模、设备数或资源总量，具体语义由本节上下文给出。
- 小写字母和希腊字母表示权重、比例、阈值、延迟或拟合参数；下标用于区分样本、设备、专家或时间步。
- 等号给出机制关系，近似号表示课程中的资源估算；实际系统还要用实测常数校准。

¹⁰视频画面时间区间：00:32:45–00:33:00。

$$G_4 = \frac{B_4}{C_4 + \lambda R_4 + \epsilon}$$

- B_4 是本节方法带来的质量、吞吐或覆盖收益。
- C_4 是新增计算、显存、通信、数据或标注成本。
- R_4 是错误、偏差、失稳或不可复现风险； λ 表示对风险的惩罚强度。

$$C_4 \leq C_{\max}, \quad R_4 \leq R_{\max}, \quad Q_4 \geq Q_{\min}$$

- $C_{\max}$ 和 $R_{\max}$ 分别是资源预算与风险上限。
- $Q_{\min}$ 是必须守住的质量、正确率或服务水平下界。
- 这组约束是本笔记用于复现实验决策的分析框架，并非把课程中的所有方法压缩为单一分数。

5.1 CUDA 与 Triton 的抽象层次比较

核心判断。“CUDA 与 Triton 的抽象层次比较”是“从 CUDA 到 Triton”中的一个可检验设计选择，而不是只需记忆的产品名。先用张量形状推出程序实例和掩码，再决定分块、向量化和片上复用；基准必须预热、同步并与高精度参考实现逐元素比较。

复现与诊断。实验记录至少包含输入形状或数据范围、关键状态位置、阈值与精度、资源测量和质量基线。先检查直接中间量，再检查最终指标；若最终分数改变而中间量没有按机制预测变化，应优先怀疑实现、基准或数据泄漏。第 11 个图像证据在本讲 25 个主题中的作用，是把这一判断固定到可回看的原始画面和时间区间。

应记住的判断

围绕“CUDA 与 Triton 的抽象层次比较”，至少记录直接机制指标、最终质量指标和一个反事实基线。若只给最终分数而没有中间量，就无法区分真正的机制收益、额外预算和测量误差。

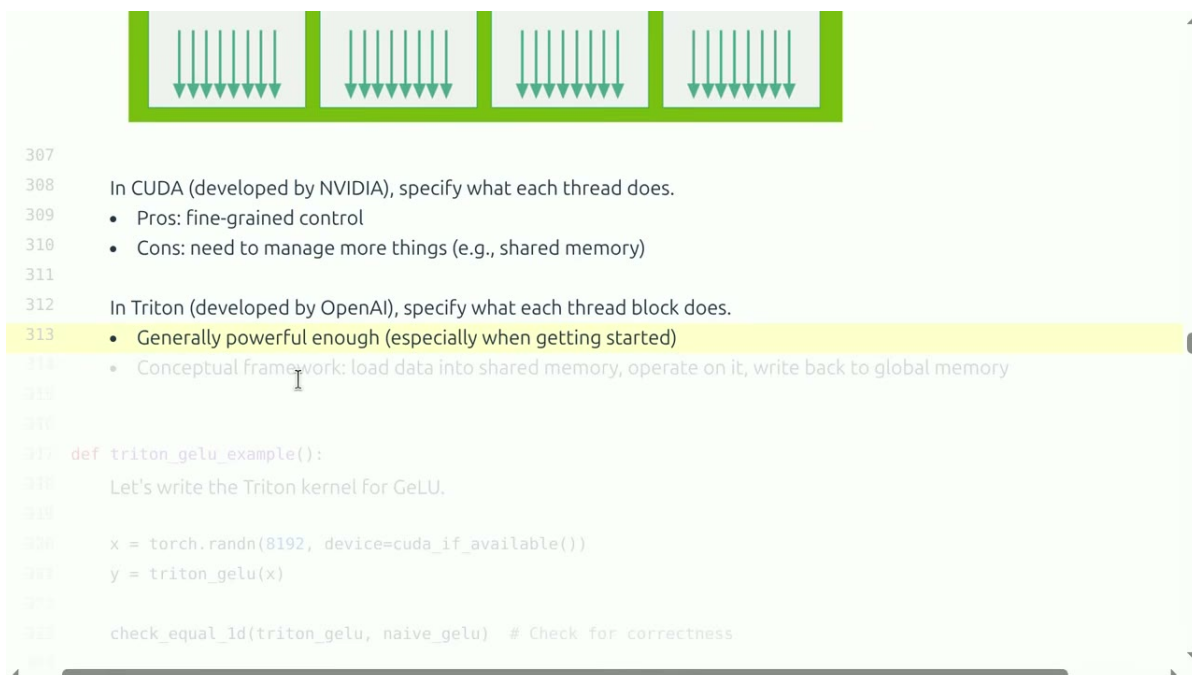


图 11: CUDA 与 Triton 的抽象层次比较¹¹

5.2 Triton 程序实例与 block size 的启动语义

核心判断。“Triton 程序实例与 block size 的启动语义”是“从 CUDA 到 Triton”中的一个可检验设计选择，而不是只需记忆的产品名。先用张量形状推出程序实例和掩码，再决定分块、向量化和片上复用；基准必须预热、同步并与高精度参考实现逐元素比较。

复现与诊断。实验记录至少包含输入形状或数据范围、关键状态位置、阈值与精度、资源测量和质量基线。先检查直接中间量，再检查最终指标；若最终分数改变而中间量没有按机制预测变化，应优先怀疑实现、基准或数据泄漏。第 12 个图像证据在本讲 25 个主题中的作用，是把这一判断固定到可回看的原始画面和时间区间。

复现实验记录

围绕“Triton 程序实例与 block size 的启动语义”，至少记录直接机制指标、最终质量指标和一个反事实基线。若只给最终分数而没有中间量，就无法区分真正的机制收益、额外预算和测量误差。

¹¹视频画面时间区间：00:38:45–00:39:00。

```
343     y = torch.empty_like(x)
    num_elements = 8192
    # Determine grid (elements divided into blocks)
    num_blocks = 8
    # | T T T T T T T T | T T T T T T T T | T T T T T T T T | T T T T T T T T |
    # | Block 0 | Block 1 | Block 2 | Block 3 |
348     num_elements = x.numel()
349     BLOCK_SIZE = 1024 # Number of threads
350     num_blocks = triton.cdiv(num_elements, BLOCK_SIZE)
351
352     # Launch the kernel
353     kernel = triton.gelu_kernel[(num_blocks,)](x, y, num_elements, BLOCK_SIZE=BLOCK_SIZE)
354
355     # Write out PTX (look at this later)
356     output_ptx("triton_gelu", kernel)
357
358     return y
359
360 @triton.jit
361 def triton_gelu_kernel(x_ptr, y_ptr, num_elements, BLOCK_SIZE: tl.constexpr):
362     # Input starts at 'x_ptr'
363     # Output starts at 'y_ptr'
```

图 12: Triton 程序实例与 block size 的启动语义¹²

5.3 Triton 内核中的程序编号掩码和向量化加载

核心判断。“Triton 内核中的程序编号掩码和向量化加载”是“从 CUDA 到 Triton”中的一个可检验设计选择，而不是只需记忆的产品名。低精度同时改变表示范围、舍入误差和搬运字节数；需要记录尺度粒度、累加精度、异常值处理以及端到端质量，而非只记录位宽。

复现与诊断。实验记录至少包含输入形状或数据范围、关键状态位置、阈值与精度、资源测量和质量基线。先检查直接中间量，再检查最终指标；若最终分数改变而中间量没有按机制预测变化，应优先怀疑实现、基准或数据泄漏。第 13 个图像证据在本讲 25 个主题中的作用，是把这一判断固定到可回看的原始画面和时间区间。

常见失效模式

围绕“Triton 内核中的程序编号掩码和向量化加载”，至少记录直接机制指标、最终质量指标和一个反事实基线。若只给最终分数而没有中间量，就无法区分真正的机制收益、额外预算和测量误差。

¹²视频画面时间区间：00:41:45–00:42:00。

```

360
361 @triton.jit
362 def triton_gelu_kernel(x_ptr, y_ptr, num_elements, BLOCK_SIZE: tl.constexpr):
363     # Input starts at `x_ptr`
364     # Output starts at `y_ptr`
365
366     # | T T T T T T T T | T T T T T T T T | T T T T T T T T | T T T T T T T T |
367     # |   Block 0       |   Block 1       |   Block 2       |   Block 3       |
368
369     pid = tl.program_id(axis=0)      # Identifies the block
370     start = pid * BLOCK_SIZE         # Starting index of this block
371
372     # Indices where this thread block should operate
373     offsets = start + tl.arange(0, BLOCK_SIZE)
374
375     # Don't read/write past the end of the tensor
376     mask = offsets < num_elements
377
378     # Read
379     x = tl.load(x_ptr + offsets, mask=mask)
380

```

图 13: Triton 内核中的程序编号掩码和向量化加载¹³

5.4 本章小结

“从 CUDA 到 Triton”的关键不是记住若干实现名称，而是建立选择顺序：先确定目标和状态边界，再核算计算、存储、通信或数据成本，随后检查本节公式的假设，最后用原始画面对应的中间量验证因果链。没有预热与同步的基准、越界掩码错误、bank conflict 和过大的资源占用都会制造看似更快但不可复现的结果。

6 融合 softmax

本节路线

本节依次处理：高级内核生成的 PTX 代码与编译边界、朴素逐行 softmax 的多次全局访存、融合 Triton softmax 减少中间张量往返。它们共享一个资源与信息流边界，因而必须联合阅读，不能把其中某张图当成独立结论。

$$y_i = \frac{\exp(x_i - \max_j x_j)}{\sum_j \exp(x_j - \max_k x_k)}$$

- 公式中的大写字母表示张量规模、样本规模、设备数或资源总量，具体语义由本节上下文给出。
- 小写字母和希腊字母表示权重、比例、阈值、延迟或拟合参数；下标用于区分样本、设备、专家或时间步。
- 等号给出机制关系，近似号表示课程中的资源估算；实际系统还要用实测常数校准。

¹³视频画面时间区间：00:44:45–00:45:00。

$$G_5 = \frac{B_5}{C_5 + \lambda R_5 + \epsilon}$$

- B_5 是本节方法带来的质量、吞吐或覆盖收益。
- C_5 是新增计算、显存、通信、数据或标注成本。
- R_5 是错误、偏差、失稳或不可复现风险； λ 表示对风险的惩罚强度。

$$C_5 \leq C_{\max}, \quad R_5 \leq R_{\max}, \quad Q_5 \geq Q_{\min}$$

- $C_{\max}$ 和 $R_{\max}$ 分别是资源预算与风险上限。
- $Q_{\min}$ 是必须守住的质量、正确率或服务水平下界。
- 这组约束是本笔记用于复现实验决策的分析框架，并非把课程中的所有方法压缩为单一分数。

6.1 高级内核生成的 PTX 代码与编译边界

核心判断。“高级内核生成的 PTX 代码与编译边界”是“融合 softmax”中的一个可检验设计选择，而不是只需记忆的产品名。先用张量形状推出程序实例和掩码，再决定分块、向量化和片上复用；基准必须预热、同步并与高精度参考实现逐元素比较。

复现与诊断。实验记录至少包含输入形状或数据范围、关键状态位置、阈值与精度、资源测量和质量基线。先检查直接中间量，再检查最终指标；若最终分数改变而中间量没有按机制预测变化，应优先怀疑实现、基准或数据泄漏。第 14 个图像证据在本讲 25 个主题中的作用，是把这一判断固定到可回看的原始画面和时间区间。

应记住的判断

围绕“高级内核生成的 PTX 代码与编译边界”，至少记录直接机制指标、最终质量指标和一个反事实基线。若只给最终分数而没有中间量，就无法区分真正的机制收益、额外预算和测量误差。

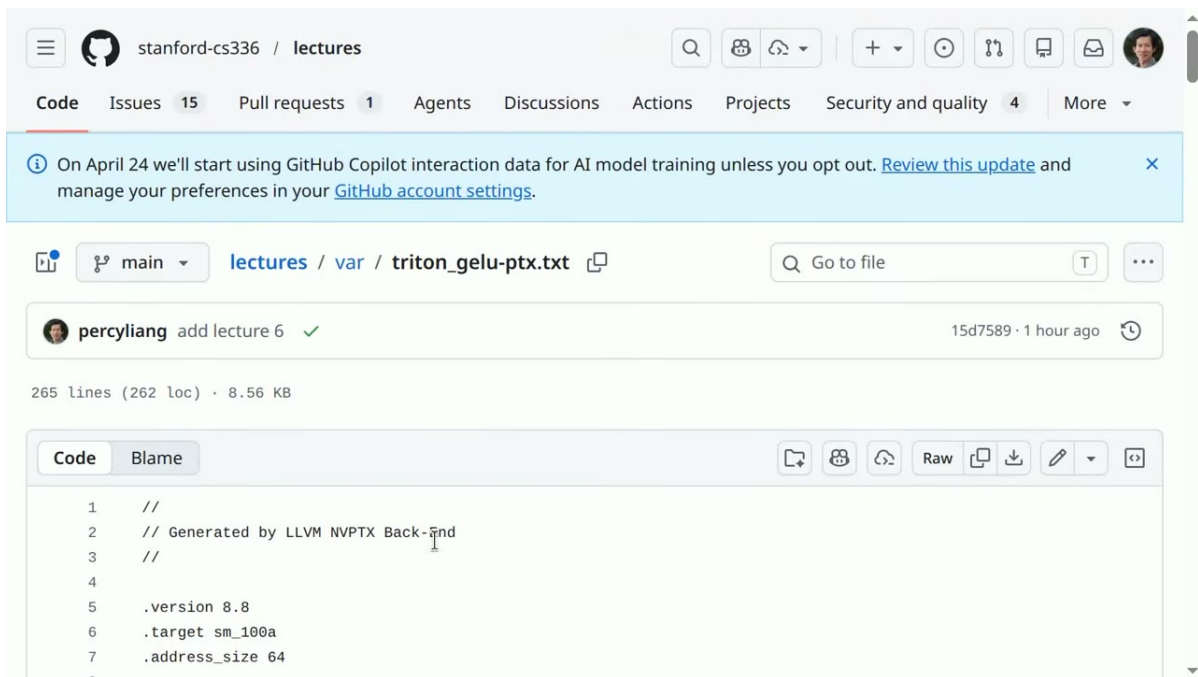


图 14: 高级内核生成的 PTX 代码与编译边界¹⁴

6.2 朴素逐行 softmax 的多次全局访存

核心判断。“朴素逐行 softmax 的多次全局访存”是“融合 softmax”中的一个可检验设计选择，而不是只需记忆的产品名。先用张量形状推出程序实例和掩码，再决定分块、向量化和片上复用；基准必须预热、同步并与高精度参考实现逐元素比较。

复现与诊断。实验记录至少包含输入形状或数据范围、关键状态位置、阈值与精度、资源测量和质量基线。先检查直接中间量，再检查最终指标；若最终分数改变而中间量没有按机制预测变化，应优先怀疑实现、基准或数据泄漏。第 15 个图像证据在本讲 25 个主题中的作用，是把这一判断固定到可回看的原始画面和时间区间。

复现实验记录

围绕“朴素逐行 softmax 的多次全局访存”，至少记录直接机制指标、最终质量指标和一个反事实基线。若只给最终分数而没有中间量，就无法区分真正的机制收益、额外预算和测量误差。

¹⁴视频画面时间区间：00:50:45–00:51:00。

```

420 # M: number of rows, N: number of columns
421 M, N = x.shape
422
423 # Compute the max of each row (MN reads, M writes)
424 x_max = x.max(dim=1)[0]
425
426 # Subtract off the max (MN + M reads, MN writes)
427 x = x - x_max[:, None]
428
429 # Exponentiate (MN reads, MN writes)
430 numerator = torch.exp(x)
431
432 # Compute normalization constant (MN reads, M writes)
433 denominator = numerator.sum(dim=1)
434
435 # Normalize (MN reads, MN writes)
436 y = numerator / denominator[:, None]
437
438 # Total: 5MN + M reads, 3MN + 2M writes
439 # In principle, should have MN reads, MN writes (speedup of 4x!)
440 return y

```

图 15: 朴素逐行 softmax 的多次全局访存¹⁵

6.3 融合 Triton softmax 减少中间张量往返

核心判断。“融合 Triton softmax 减少中间张量往返”是“融合 softmax”中的一个可检验设计选择，而不是只需记忆的产品名。先用张量形状推出程序实例和掩码，再决定分块、向量化和片上复用；基准必须预热、同步并与高精度参考实现逐元素比较。

复现与诊断。实验记录至少包含输入形状或数据范围、关键状态位置、阈值与精度、资源测量和质量基线。先检查直接中间量，再检查最终指标；若最终分数改变而中间量没有按机制预测变化，应优先怀疑实现、基准或数据泄漏。第 16 个图像证据在本讲 25 个主题中的作用，是把这一判断固定到可回看的原始画面和时间区间。

常见失效模式

围绕“融合 Triton softmax 减少中间张量往返”，至少记录直接机制指标、最终质量指标和一个反事实基线。若只给最终分数而没有中间量，就无法区分真正的机制收益、额外预算和测量误差。

¹⁵视频画面时间区间：00:59:45–01:00:00。

```

461
462 @triton.jit
463 def triton_softmax_kernel(x_ptr, y_ptr, x_row_stride, y_row_stride, num_cols, BLOCK_SIZE: tl.constexpr):
464     assert num_cols <= BLOCK_SIZE
465
466     # Process each row independently
467     row_idx = tl.program_id(0)
468     col_offsets = tl.arange(0, BLOCK_SIZE)
469
470     # Read from global memory
471     x_start_ptr = x_ptr + row_idx * x_row_stride
472     x_ptrs = x_start_ptr + col_offsets
473     x_row = tl.load(x_ptrs, mask=col_offsets < num_cols, other=float("-inf"))
474
475     # Compute
476     x_row = x_row - tl.max(x_row, axis=0)
477     numerator = tl.exp(x_row)
478     denominator = tl.sum(numerator, axis=0)
479     y_row = numerator / denominator
480
481     # Write back to global memory
482     y_start_ptr = y_ptr + row_idx * y_row_stride

```

图 16: 融合 Triton softmax 减少中间张量往返¹⁶

6.4 本章小结

“融合 softmax”的关键不是记住若干实现名称，而是建立选择顺序：先确定目标和状态边界，再核算计算、存储、通信或数据成本，随后检查本节公式的假设，最后用原始画面对应的中间量验证因果链。没有预热与同步的基准、越界掩码错误、bank conflict 和过大的资源占用都会制造看似更快但不可复现的结果。

7 并行归约

本节路线

本节依次处理：行归约的并行拆分与树形求和、Triton 行求和内核的掩码处理、循环分块处理超长行的实现。它们共享一个资源与信息流边界，因而必须联合阅读，不能把其中某张图当成独立结论。

$$s = \sum_{i=1}^N x_i$$

- 公式中的大写字母表示张量规模、样本规模、设备数或资源总量，具体语义由本节上下文给出。
- 小写字母和希腊字母表示权重、比例、阈值、延迟或拟合参数；下标用于区分样本、设备、专家或时间步。
- 等号给出机制关系，近似号表示课程中的资源估算；实际系统还要用实测常数校准。

¹⁶视频画面时间区间：01:02:45–01:03:00。

$$G_6 = \frac{B_6}{C_6 + \lambda R_6 + \epsilon}$$

- B_6 是本节方法带来的质量、吞吐或覆盖收益。
- C_6 是新增计算、显存、通信、数据或标注成本。
- R_6 是错误、偏差、失稳或不可复现风险； λ 表示对风险的惩罚强度。

$$C_6 \leq C_{\max}, \quad R_6 \leq R_{\max}, \quad Q_6 \geq Q_{\min}$$

- $C_{\max}$ 和 $R_{\max}$ 分别是资源预算与风险上限。
- $Q_{\min}$ 是必须守住的质量、正确率或服务水平下界。
- 这组约束是本笔记用于复现实验决策的分析框架，并非把课程中的所有方法压缩为单一分数。

7.1 行归约的并行拆分与树形求和

核心判断。“行归约的并行拆分与树形求和”是“并行归约”中的一个可检验设计选择，而不是只需记忆的产品名。先用张量形状推出程序实例和掩码，再决定分块、向量化和片上复用；基准必须预热、同步并与高精度参考实现逐元素比较。

复现与诊断。实验记录至少包含输入形状或数据范围、关键状态位置、阈值与精度、资源测量和质量基线。先检查直接中间量，再检查最终指标；若最终分数改变而中间量没有按机制预测变化，应优先怀疑实现、基准或数据泄漏。第 17 个图像证据在本讲 25 个主题中的作用，是把这一判断固定到可回看的原始画面和时间区间。

应记住的判断

围绕“行归约的并行拆分与树形求和”，至少记录直接机制指标、最终质量指标和一个反事实基线。若只给最终分数而没有中间量，就无法区分真正的机制收益、额外预算和测量误差。

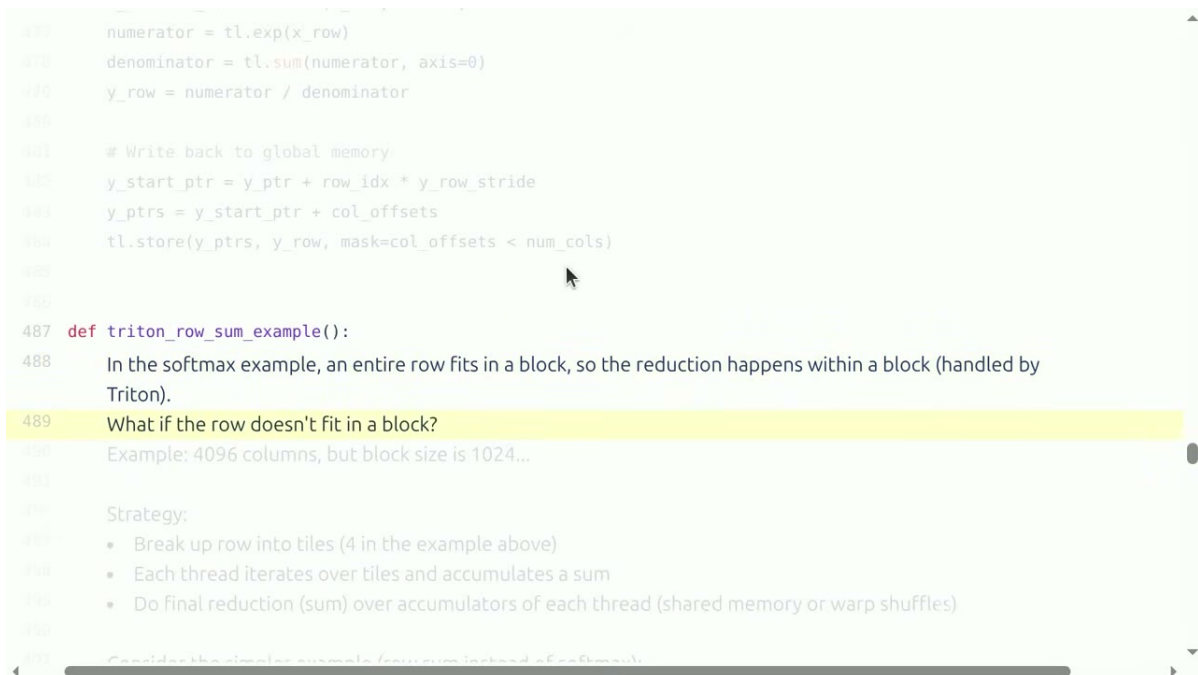


图 17: 行归约的并行拆分与树形求和¹⁷

7.2 Triton 行求和内核的掩码处理

核心判断。“Triton 行求和内核的掩码处理”是“并行归约”中的一个可检验设计选择，而不是只需记忆的产品名。先用张量形状推出程序实例和掩码，再决定分块、向量化和片上复用；基准必须预热、同步并与高精度参考实现逐元素比较。

复现与诊断。实验记录至少包含输入形状或数据范围、关键状态位置、阈值与精度、资源测量和质量基线。先检查直接中间量，再检查最终指标；若最终分数改变而中间量没有按机制预测变化，应优先怀疑实现、基准或数据泄漏。第 18 个图像证据在本讲 25 个主题中的作用，是把这一判断固定到可回看的原始画面和时间区间。

复现实验记录

围绕“Triton 行求和内核的掩码处理”，至少记录直接机制指标、最终质量指标和一个反事实基线。若只给最终分数而没有中间量，就无法区分真正的机制收益、额外预算和测量误差。

¹⁷视频画面时间区间：01:05:30–01:05:45。

```

514     return y
515
516
517 @triton.jit
518 def row_sum_kernel(x_ptr, out_ptr, N, BLOCK_SIZE: tl.constexpr):
519     row = tl.program_id(0) # Which row are we processing?
520
521     # Accumulator for each thread
522     # One row: T1 T2 T3 T4 | T1 T2 T3 T4 | T1 T2 T3 T4 (N = 12, BLOCK_SIZE = 4)
523     acc = tl.zeros([BLOCK_SIZE], dtype=tl.float32)
524
525     # Loop over tiles
526     for start in range(0, N, BLOCK_SIZE):
527         cols = start + tl.arange(0, BLOCK_SIZE)
528         mask = cols < N
529         x = tl.load(x_ptr + row * N + cols, mask=mask, other=0.0)
530         acc += x
531
532     # Final reduction from BLOCK_SIZE (all threads) to a scalar
533     result = tl.sum(acc, axis=0)
534
535     tl.store(out_ptr + row, result)

```

图 18: Triton 行求和内核的掩码处理¹⁸

7.3 循环分块处理超长行的实现

核心判断。“循环分块处理超长行的实现”是“并行归约”中的一个可检验设计选择，而不是只需记忆的产品名。共享块通过多次递归增加计算深度而不增加同等参数量；必须检查残差动力学、谱半径、梯度传播和固定 FLOPs 下的最优循环次数。

复现与诊断。实验记录至少包含输入形状或数据范围、关键状态位置、阈值与精度、资源测量和质量基线。先检查直接中间量，再检查最终指标；若最终分数改变而中间量没有按机制预测变化，应优先怀疑实现、基准或数据泄漏。第 19 个图像证据在本讲 25 个主题中的作用，是把这一判断固定到可回看的原始画面和时间区间。

常见失效模式

围绕“循环分块处理超长行的实现”，至少记录直接机制指标、最终质量指标和一个反事实基线。若只给最终分数而没有中间量，就无法区分真正的机制收益、额外预算和测量误差。

¹⁸视频画面时间区间：01:08:30–01:08:45。


```

515
516
517 @triton.jit
518 def row_sum_kernel(x_ptr, out_ptr, N, BLOCK_SIZE: tl.constexpr):
519     row = tl.program_id(0) # Which row are we processing?
520
521     # Accumulator for each thread
522     # One row: T1 T2 T3 T4 | T1 T2 T3 T4 | T1 T2 T3 T4 (N = 12, BLOCK_SIZE = 4)
523     acc = tl.zeros([BLOCK_SIZE], dtype=tl.float32)
524
525     # Loop over tiles
526     for start in range(0, N, BLOCK_SIZE):
527         cols = start + tl.arange(0, BLOCK_SIZE)
528         mask = cols < N
529         x = tl.load(x_ptr + row * N + cols, mask=mask, other=0.0)
530         acc += x
531
532     # Final reduction from BLOCK_SIZE (all threads) to a scalar
533     result = tl.sum(acc, axis=0)
534
535     tl.store(out_ptr + row, result)
536

```

图 19: 循环分块处理超长行的实现¹⁹

7.4 本章小结

“并行归约”的关键不是记住若干实现名称，而是建立选择顺序：先确定目标和状态边界，再核算计算、存储、通信或数据成本，随后检查本节公式的假设，最后用原始画面对应的中间量验证因果链。没有预热与同步的基准、越界掩码错误、bank conflict 和过大的资源占用都会制造看似更快但不可复现的结果。

8 分块矩阵乘

本节路线

本节依次处理：朴素矩阵乘的程序结构与瓶颈、矩阵乘分块在片上复用输入、Triton 分块矩阵乘的指针和累加器。它们共享一个资源与信息流边界，因而必须联合阅读，不能把其中某张图当成独立结论。

$$C_{ij} = \sum_{k=1}^K A_{ik} B_{kj}$$

- 公式中的大写字母表示张量规模、样本规模、设备数或资源总量，具体语义由本节上下文给出。
- 小写字母和希腊字母表示权重、比例、阈值、延迟或拟合参数；下标用于区分样本、设备、专家或时间步。
- 等号给出机制关系，近似号表示课程中的资源估算；实际系统还要用实测常数校准。

¹⁹视频画面时间区间：01:11:30–01:11:45。

$$G_7 = \frac{B_7}{C_7 + \lambda R_7 + \epsilon}$$

- B_7 是本节方法带来的质量、吞吐或覆盖收益。
- C_7 是新增计算、显存、通信、数据或标注成本。
- R_7 是错误、偏差、失稳或不可复现风险； λ 表示对风险的惩罚强度。

$$C_7 \leq C_{\max}, \quad R_7 \leq R_{\max}, \quad Q_7 \geq Q_{\min}$$

- $C_{\max}$ 和 $R_{\max}$ 分别是资源预算与风险上限。
- $Q_{\min}$ 是必须守住的质量、正确率或服务水平下界。
- 这组约束是本笔记用于复现实验决策的分析框架，并非把课程中的所有方法压缩为单一分数。

8.1 朴素矩阵乘的程序结构与瓶颈

核心判断。“朴素矩阵乘的程序结构与瓶颈”是“分块矩阵乘”中的一个可检验设计选择，而不是只需记忆的产品名。先用张量形状推出程序实例和掩码，再决定分块、向量化和片上复用；基准必须预热、同步并与高精度参考实现逐元素比较。

复现与诊断。实验记录至少包含输入形状或数据范围、关键状态位置、阈值与精度、资源测量和质量基线。先检查直接中间量，再检查最终指标；若最终分数改变而中间量没有按机制预测变化，应优先怀疑实现、基准或数据泄漏。第 20 个图像证据在本讲 25 个主题中的作用，是把这一判断固定到可回看的原始画面和时间区间。

应记住的判断

围绕“朴素矩阵乘的程序结构与瓶颈”，至少记录直接机制指标、最终质量指标和一个反事实基线。若只给最终分数而没有中间量，就无法区分真正的机制收益、额外预算和测量误差。

```

541 b = torch.randn(1024, 1024, device=cuda_if_available())
542 c = naive_matmul_relu(a, b)
543
544 How should we build a matmul kernel?
545
546 |           k           n
547 | [ A1 A2 A3 ] [ B1 B2 B3 ] [ C1 C2 C3 ]
548 | m [ A4 A5 A6 ] * k [ B4 B5 B6 ] = [ C4 C5 C6 ]
549 | [ A7 A8 A9 ] [ B7 B8 B9 ] [ C7 C8 C9 ]
550
551 Naive approach:
552 Fix any (m, n).
553 For each k:
554   • Read A[m, k] and B[k, n] from HBM.
555   • Multiply and accumulate.
556 Write result to C[m, n] in HBM.
557
558 Bottleneck:  $\frac{1}{3}MN$  reads,  $\frac{1}{3}MN$  writes
559 Arithmetic intensity:  $O(1)$ 
560
561 Computing C4 and C5 both need A4, A5, A6.
562 Can we read A4, A5, A6 from HBM once to compute both?

```

图 20: 朴素矩阵乘的程序结构与瓶颈²⁰

8.2 矩阵乘分块在片上复用输入

核心判断。“矩阵乘分块在片上复用输入”是“分块矩阵乘”中的一个可检验设计选择，而不是只需记忆的产品名。先用张量形状推出程序实例和掩码，再决定分块、向量化和片上复用；基准必须预热、同步并与高精度参考实现逐元素比较。

复现与诊断。实验记录至少包含输入形状或数据范围、关键状态位置、阈值与精度、资源测量和质量基线。先检查直接中间量，再检查最终指标；若最终分数改变而中间量没有按机制预测变化，应优先怀疑实现、基准或数据泄漏。第 21 个图像证据在本讲 25 个主题中的作用，是把这一判断固定到可回看的原始画面和时间区间。

复现实验记录

围绕“矩阵乘分块在片上复用输入”，至少记录直接机制指标、最终质量指标和一个反事实基线。若只给最终分数而没有中间量，就无法区分真正的机制收益、额外预算和测量误差。

²⁰视频画面时间区间：01:14:30–01:14:45。

568 • This yields the idealized $O(N)$ arithmetic intensity from before.
 569 • However, A and B are usually too large to fit in shared memory.
 570
 571 **Tiling:**
 572
 573

Matrix A • Matrix B = Matrix C

tile size T matrix size N

Outer loop over tiles
 Current tile in outer loop
 Inner loop over elements
 Current element in inner loop
 Temporary result tile

574 Key idea: divide the matrix C into output tiles (thread blocks).
 575 Fix an output tile in C.
 576 For each pair of (row tile of A, column tile of B):
 577 • Load the corresponding A tile and B tile from HBM into shared memory.
 578 • Perform matrix multiplication on the tiles.

图 21: 矩阵乘分块在片上复用输入²¹

8.3 Triton 分块矩阵乘的指针和累加器

核心判断。“Triton 分块矩阵乘的指针和累加器”是“分块矩阵乘”中的一个可检验设计选择，而不是只需记忆的产品名。先用张量形状推出程序实例和掩码，再决定分块、向量化和片上复用；基准必须预热、同步并与高精度参考实现逐元素比较。

复现与诊断。实验记录至少包含输入形状或数据范围、关键状态位置、阈值与精度、资源测量和质量基线。先检查直接中间量，再检查最终指标；若最终分数改变而中间量没有按机制预测变化，应优先怀疑实现、基准或数据泄漏。第 22 个图像证据在本讲 25 个主题中的作用，是把这一判断固定到可回看的原始画面和时间区间。

常见失效模式

围绕“Triton 分块矩阵乘的指针和累加器”，至少记录直接机制指标、最终质量指标和一个反事实基线。若只给最终分数而没有中间量，就无法区分真正的机制收益、额外预算和测量误差。

²¹视频画面时间区间：01:17:30–01:17:45。

```

639     stride_am, stride_ak, # How to navigate a
640     stride_bk, stride_bn, # How to navigate b
641     stride_cm, stride_cn, # How to navigate c
642     BLOCK_M: tl.constexpr,
643     BLOCK_N: tl.constexpr,
644     BLOCK_K: tl.constexpr,
645 ):
646     # We are working on the (m, n)-th tile
647     pid_m = tl.program_id(0)
648     pid_n = tl.program_id(1)
649
650     # Indices
651     indices_m = pid_m * BLOCK_M + tl.arange(0, BLOCK_M) # Row indices of a [BLOCK_M]
652     indices_n = pid_n * BLOCK_N + tl.arange(0, BLOCK_N) # Column indices of b [BLOCK_N]
653     indices_k = tl.arange(0, BLOCK_K) # Row indices of a = column indices of b [BLOCK_K]
654
655     # Initial matrix of pointers of a and b
656     a_ptrs = a_ptr + indices_m[:, None] * stride_am + indices_k[None, :] * stride_ak # [BLOCK_M, BLOCK_K]
657     b_ptrs = b_ptr + indices_k[:, None] * stride_bk + indices_n[None, :] * stride_bn # [BLOCK_K, BLOCK_N]
658
659     acc = tl.zeros([BLOCK_M, BLOCK_N], dtype=tl.float32)
660

```

图 22: Triton 分块矩阵乘的指针和累加器²²

8.4 本章小结

“分块矩阵乘”的关键不是记住若干实现名称，而是建立选择顺序：先确定目标和状态边界，再核算计算、存储、通信或数据成本，随后检查本节公式的假设，最后用原始画面对应的中间量验证因果链。没有预热与同步的基准、越界掩码错误、bank conflict 和过大的资源占用都会制造看似更快但不可复现的结果。

9 编译器 XLA 与优化边界

本节路线

本节依次处理：从正确性到吞吐的内核优化检查表、线程块按 wave 调度与占用率限制、Triton 融合 GELU 的代码练习入口。它们共享一个资源与信息流边界，因而必须联合阅读，不能把其中某张图当成独立结论。

$$S = \frac{T_{\text{baseline}}}{T_{\text{optimized}}}$$

- 公式中的大写字母表示张量规模、样本规模、设备数或资源总量，具体语义由本节上下文给出。
- 小写字母和希腊字母表示权重、比例、阈值、延迟或拟合参数；下标用于区分样本、设备、专家或时间步。
- 等号给出机制关系，近似号表示课程中的资源估算；实际系统还要用实测常数校准。

²²视频画面时间区间：01:20:30–01:20:45。

$$G_8 = \frac{B_8}{C_8 + \lambda R_8 + \epsilon}$$

- B_8 是本节方法带来的质量、吞吐或覆盖收益。
- C_8 是新增计算、显存、通信、数据或标注成本。
- R_8 是错误、偏差、失稳或不可复现风险； λ 表示对风险的惩罚强度。

$$C_8 \leq C_{\max}, \quad R_8 \leq R_{\max}, \quad Q_8 \geq Q_{\min}$$

- $C_{\max}$ 和 $R_{\max}$ 分别是资源预算与风险上限。
- $Q_{\min}$ 是必须守住的质量、正确率或服务水平下界。
- 这组约束是本笔记用于复现实验决策的分析框架，并非把课程中的所有方法压缩为单一分数。

9.1 从正确性到吞吐的内核优化检查表

核心判断。“从正确性到吞吐的内核优化检查表”是“编译器 XLA 与优化边界”中的一个可检验设计选择，而不是只需记忆的产品名。先用张量形状推出程序实例和掩码，再决定分块、向量化和片上复用；基准必须预热、同步并与高精度参考实现逐元素比较。

复现与诊断。实验记录至少包含输入形状或数据范围、关键状态位置、阈值与精度、资源测量和质量基线。先检查直接中间量，再检查最终指标；若最终分数改变而中间量没有按机制预测变化，应优先怀疑实现、基准或数据泄漏。第 23 个图像证据在本讲 25 个主题中的作用，是把这一判断固定到可回看的原始画面和时间区间。

应记住的判断

围绕“从正确性到吞吐的内核优化检查表”，至少记录直接机制指标、最终质量指标和一个反事实基线。若只给最终分数而没有中间量，就无法区分真正的机制收益、额外预算和测量误差。

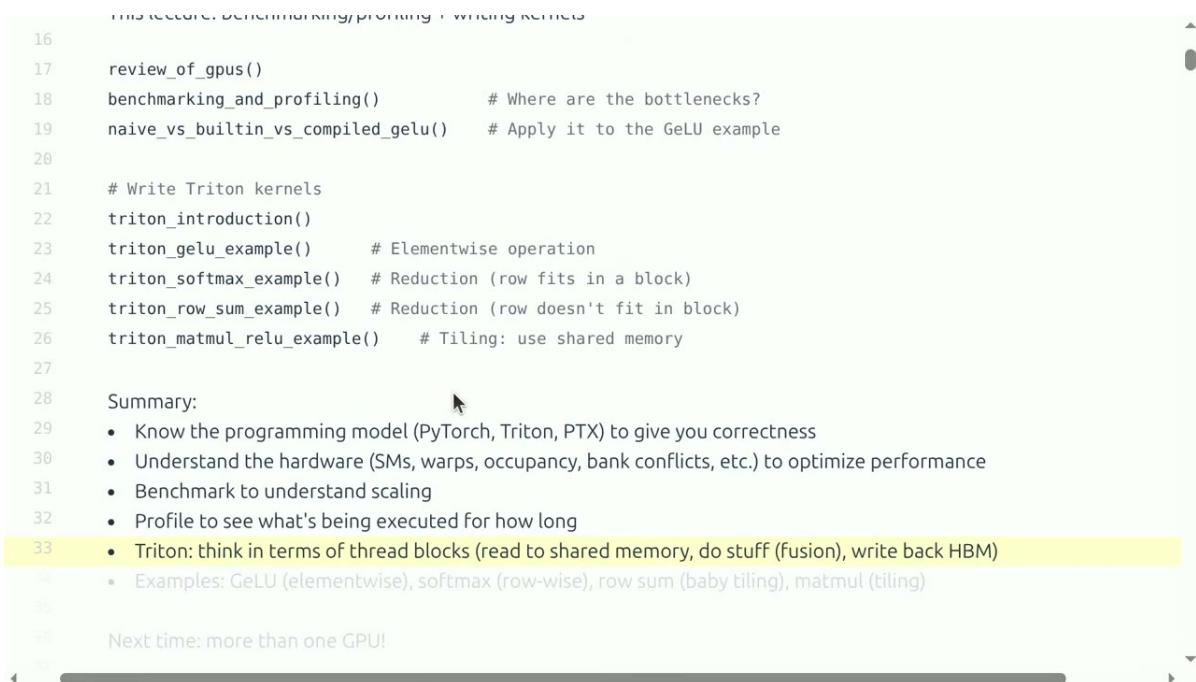


图 23: 从正确性到吞吐的内核优化检查表²³

9.2 线程块按 wave 调度与占用率限制

核心判断。“线程块按 wave 调度与占用率限制”是“编译器 XLA 与优化边界”中的一个可检验设计选择，而不是只需记忆的产品名。把该主题拆成输入、变换、持久状态和输出四部分，再分别记录直接机制指标、资源成本和最终质量；若中间量不按预测变化，应优先检查实现和数据边界。

复现与诊断。实验记录至少包含输入形状或数据范围、关键状态位置、阈值与精度、资源测量和质量基线。先检查直接中间量，再检查最终指标；若最终分数改变而中间量没有按机制预测变化，应优先怀疑实现、基准或数据泄漏。第 24 个图像证据在本讲 25 个主题中的作用，是把这一判断固定到可回看的原始画面和时间区间。

复现实验记录

围绕“线程块按 wave 调度与占用率限制”，至少记录直接机制指标、最终质量指标和一个反事实基线。若只给最终分数而没有中间量，就无法区分真正的机制收益、额外预算和测量误差。

²³视频画面时间区间：01:23:30–01:23:45。

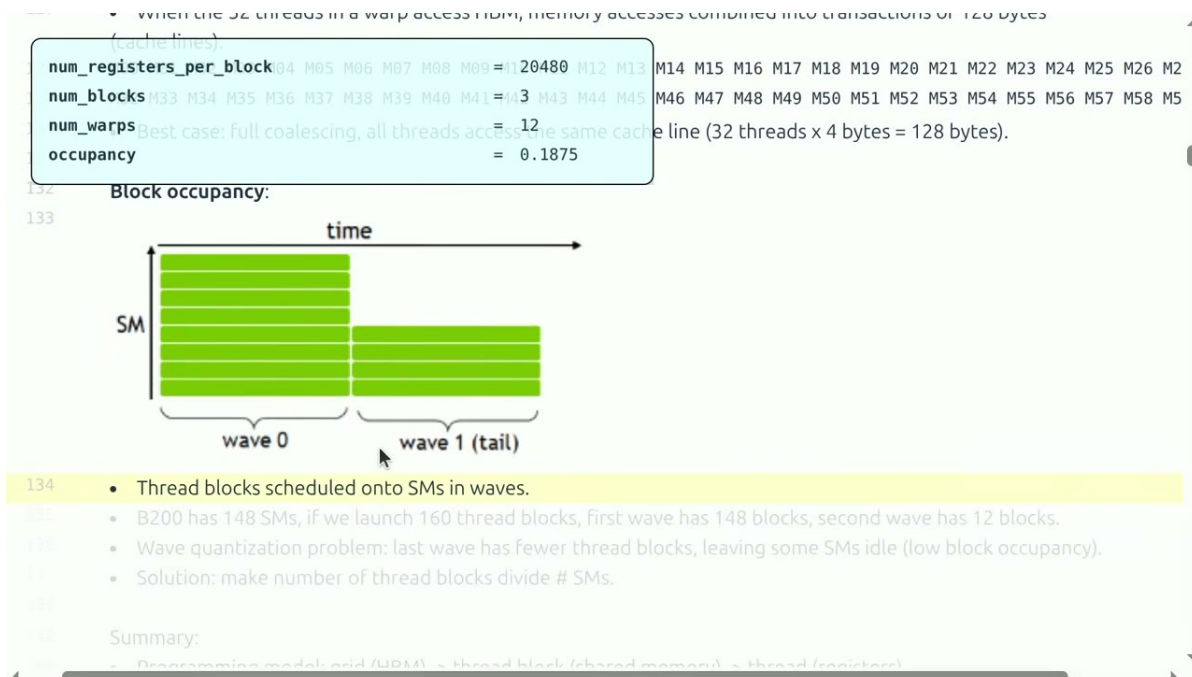


图 24: 线程块按 wave 调度与占用率限制²⁴

9.3 Triton 融合 GELU 的代码练习入口

核心判断。“Triton 融合 GELU 的代码练习入口”是“编译器 XLA 与优化边界”中的一个可检验设计选择，而不是只需记忆的产品名。先用张量形状推出程序实例和掩码，再决定分块、向量化和片上复用；基准必须预热、同步并与高精度参考实现逐元素比较。

复现与诊断。实验记录至少包含输入形状或数据范围、关键状态位置、阈值与精度、资源测量和质量基线。先检查直接中间量，再检查最终指标；若最终分数改变而中间量没有按机制预测变化，应优先怀疑实现、基准或数据泄漏。第 25 个图像证据在本讲 25 个主题中的作用，是把这一判断固定到可回看的原始画面和时间区间。

常见失效模式

围绕“Triton 融合 GELU 的代码练习入口”，至少记录直接机制指标、最终质量指标和一个反事实基线。若只给最终分数而没有中间量，就无法区分真正的机制收益、额外预算和测量误差。

²⁴视频画面时间区间：00:18:15–00:18:30。

```
13 def main():
14     Last lecture: high-level overview of GPUs and performance
15     This lecture: benchmarking/profiling + writing kernels
16
17     review_of_gpus()
18     benchmarking_and_profiling()      # Where are the bottlenecks?
19     naive_vs_builtin_vs_compiled_gelu() # Apply it to the GeLU example
20
21     # Write Triton kernels
22     triton_introduction()
23     triton_gelu_example()              # Elementwise operation
24     triton_softmax_example()           # Reduction (row fits in a block)
25     triton_row_sum_example()           # Reduction (row doesn't fit in block)
26     triton_matmul_relu_example()       # Tiling: use shared memory
27
28     Summary:
29     * Know the programming model (PyTorch, Triton, PTX) to give you correctness
30     * Understand the hardware (SMs, warps, occupancy, bank conflicts, etc.) to optimize performance
31     * Benchmark to understand scaling
32     * Profile to see what's being executed for how long
33     * Triton: think in terms of thread blocks (read to shared memory, do stuff (fusion), write back HBM)
```

图 25: Triton 融合 GELU 的代码练习入口²⁵

9.4 本章小结

“编译器 XLA 与优化边界”的关键不是记住若干实现名称，而是建立选择顺序：先确定目标和状态边界，再核算计算、存储、通信或数据成本，随后检查本节公式的假设，最后用原始画面对应的中间量验证因果链。没有预热与同步的基准、越界掩码错误、bank conflict 和过大的资源占用都会制造看似更快但不可复现的结果。

10 代码化复现：最小机制

下面的片段保留本讲最核心的程序结构，用于把公式和系统过程落到可执行对象。它不是生产实现；实际复现还需补充张量形状断言、数值精度、同步、随机种子、数据版本和端到端测试。

Listing 1: Lecture 6 机制伪代码

```
1 @triton.jit
2 def row_softmax(x, y, n: tl.constexpr, BLOCK: tl.constexpr):
3     row = tl.program_id(0)
4     col = tl.arange(0, BLOCK)
5     z = tl.load(x + row * n + col, mask=col < n, other=-float("inf"))
6     z = z - tl.max(z, axis=0)
7     out = tl.exp(z) / tl.sum(tl.exp(z), axis=0)
8     tl.store(y + row * n + col, out, mask=col < n)
```

²⁵视频画面时间区间：00:54:45–00:55:00。

复现检查单

先写正确性基线，再记录资源基线；每次只改变一个机制；同时保存直接中间量和最终指标；对随机算法报告多次运行；对数据与评测保存不可变版本标识。

10.1 本章小结

代码的价值在于暴露状态、顺序和边界。若一个概念无法写成输入、变换、状态、输出和断言，它通常还没有被真正理解。

11 总结与延伸

一页压缩

本讲的主问题是：怎样从算子语义出发，写出既正确又能利用 GPU 层级并行和存储复用的 CUDA/Triton 内核？统一答案是：内核优化是一条可验证链：定义索引与掩码，测量真实时间，定位访存或计算瓶颈，再改变映射和融合方式。

第一，优先理解机制而不是产品名。模型、硬件、数据和评测不断更新，但张量形状、状态位置、目标函数、通信边界和测量误差仍然是稳定的分析对象。第二，所有优化都要写出代价守恒：减少某一种成本通常会增加另一种成本或风险。第三，把小规模实验推广到大规模之前，必须检查瓶颈是否改变、数据分布是否改变、实现常数是否仍在同一数量级。第四，最可靠的课程笔记应允许读者沿时间轴注回到原视频，并沿官方材料找到公式或代码出处。

开放问题

没有预热与同步的基准、越界掩码错误、bank conflict 和过大的资源占用都会制造看似更快但不可复现的结果。后续实验应把这一风险写成可观测量和停止条件，而不是留在讨论部分。

11.1 本章小结

完成本讲后，读者应能用统一语言解释每个主题的输入、状态、变换、输出、资源成本和失败模式，并据此设计一个含基线、消融、误差条和证据记录的最小实验。

12 资料来源与审计边界

- 课程主页：<https://cs336.stanford.edu/>。
- 视频：<https://www.youtube.com/watch?v=xnDHaNUvHBg>。
- Stanford 官方材料：lecture_06.py；客座讲座若无官方文件则以视频和字幕为主。
- 本地证据：audio.srt、figure_manifest.tsv、figure_verification.txt、teaching_atoms.tsv、numerical_claims.tsv。

12.1 本章小结

本笔记覆盖播放列表中的本讲内容与课程主页当日可取得的官方材料。没有官方讲义的客座讲座不会被伪装成有讲义；讲者口头推测、本笔记分析框架和官方公式在文字中分别标明。